

BÁO CÁO THỰC HÀNH SRE: PRODUCTION BACKEND LAB - FULL MODE

Ngày lập báo cáo: 11/06/2026 **Đối tượng review:** Mentor SRE **Phạm vi:** Khởi động, kiểm tra chức năng, kiểm tra hiệu năng, observability, chaos testing, Elasticsearch backup/restore và đánh giá rủi ro **Mã nguồn nền:** commit adb457a cùng các thay đổi lab chưa commit trong workspace

1. Tóm tắt điều hành

Production Backend Lab mô phỏng một hệ thống xử lý đơn hàng bất đồng bộ. API nhận request, kiểm tra rate limit và idempotency trong Redis, publish persistent message vào RabbitMQ quorum queue, sau đó consumer xử lý và index kết quả vào Elasticsearch. Log JSON của API và consumer được Logstash ingest vào Elasticsearch để hỗ trợ trace.

Full mode đã được khởi động và kiểm tra thành công với:

- RabbitMQ cluster 3 node, quorum queue, HAProxy và publisher confirms.
- Redis primary, một replica, một Sentinel, AOF và RDB.
- Elasticsearch một node, index nghiệp vụ, index log, slow logs và filesystem snapshot repository.
- API server, consumer worker và Logstash.
- Các bài kiểm tra chức năng, tải cao, RabbitMQ persistence trade-off và snapshot backup.

Trạng thái tại thời điểm lập báo cáo:

Hạng mục	Trạng thái	Bảng chứng chính
API readiness	Đạt	Redis, Elasticsearch và RabbitMQ đều true
RabbitMQ cluster	Đạt	3/3 node running, không alarm, không network partition
RabbitMQ order queues	Đạt	orders.main.q, orders.retry.q, orders.dlq đều là quorum queue
Redis replication	Đạt	Primary có 1 replica online, replication lag quan sát là 1
Redis Sentinel	Đạt trong phạm vi lab	Sentinel nhận diện primary và 1 replica
Elasticsearch nghiệp vụ	Đạt	Index orders green
Elasticsearch cluster	Cần lưu ý	Cluster yellow do backing index log có 1 replica nhưng chỉ có 1 node
Elasticsearch snapshot	Đạt	Snapshot smoke-test ở trạng thái SUCCESS
Performance tooling	Đạt	Có HTTP load, search load, full-stack observer và RabbitMQ persistence benchmark

Kết luận: lab phù hợp để trình diễn các nguyên lý SRE và backend production. Lab chưa phải kiến trúc production hoàn chỉnh vì Elasticsearch chỉ có một node, Redis Sentinel chỉ có một tiến trình và ứng dụng chưa sử dụng Sentinel-aware client.

2. Mục tiêu và tiêu chí đánh giá

2.1 Mục tiêu

1. Xác minh full stack khởi động ổn định và các dependency sẵn sàng.
2. Xác minh luồng xử lý đơn hàng end-to-end.
3. Kiểm tra idempotency, rate limit, retry, dead letter và traceability.
4. Tạo tải đồng thời để xác định điểm bão hòa và bottleneck.
5. Quan sát riêng RabbitMQ, Redis và Elasticsearch khi cao tải.
6. Đo trade-off giữa transient và persistent RabbitMQ message.
7. Cấu hình Elasticsearch slow logs và kiểm tra snapshot backup.
8. Thực hành lỗi thành phần và đánh giá khả năng phục hồi.

2.2 Nguyên tắc đánh giá

- Không đánh giá hệ thống chỉ bằng throughput.
- Luôn ghi nhận đồng thời error rate, p50, p95, p99, backlog và resource pressure.
- Tăng tải theo từng bước và giữ payload ổn định giữa các lần chạy.
- Chỉ kết luận bottleneck khi dấu hiệu lặp lại và tương quan với metric phía server.
- Phân biệt kết quả smoke test với benchmark có kiểm soát.

3. Kiến trúc hệ thống

3.1 Luồng nghiệp vụ

Client

- > API Server
 - > Redis: rate limit, idempotency, job state
 - > HAProxy
 - > RabbitMQ cluster 3 node
 - > orders.main.q (quorum)
 - > Consumer Worker
 - > Redis: cập nhật job state
 - > Elasticsearch: index order

API Server + Consumer Worker

- > JSON log file
- > Logstash
- > Elasticsearch logs-app-*

3.2 Thành phần full mode

Thành phần	Vai trò	Đặc điểm chính
API server	REST API nhận order	Publisher confirms, persistent message, readiness checks
Consumer worker	Xử lý message	Manual ACK, retry TTL, DLQ, configurable prefetch

Thành phần	Vai trò	Đặc điểm chính
RabbitMQ 1-3	Message broker cluster	Quorum queue, durable topology, Prometheus plugin
HAProxy	AMQP endpoint	Round-robin và TCP health check
Redis primary	Trạng thái tạm thời	AOF every second, RDB, allkeys-lru
Redis replica	Sao chép primary	Dùng cho bài Sentinel/failover
Redis Sentinel	Theo dõi primary	Một Sentinel phục vụ quan sát, không đủ quorum production
Elasticsearch	Business search và log search	Một node để tiết kiệm RAM
Logstash	Parse/enrich log	Đưa log ứng dụng vào Elasticsearch
Snapshot init	Chuẩn bị quyền volume	Init container, kết thúc với exit code 0

3.3 Durability và delivery semantics

- RabbitMQ exchange và queue được khai báo durable.
- Full order flow sử dụng quorum queue.
- API publish message với delivery_mode=2 và publisher confirms.
- Consumer dùng manual acknowledgement.
- Consumer chỉ ACK sau khi side effect hoàn thành hoặc retry/DLQ đã được publish.
- Hệ thống vẫn có cửa sổ xử lý trùng lặp nếu consumer hoàn tất side effect nhưng chết trước ACK. Consumer production phải idempotent.

4. Chuẩn bị môi trường

4.1 Yêu cầu

- Docker Engine 28.1.1.
- Docker Compose v2.35.1.
- Linux x86_64.
- Khoảng 5-6 GB RAM trống.
- Các port mặc định: 8000, 5672, 15672, 6379, 9200.
- Linux host cần vm.max_map_count >= 262144 cho Elasticsearch.

4.2 Chuẩn bị cấu hình

```
cd /home/sonth32/production-backend-lab
cp -n .env.example .env
sudo sysctl -w vm.max_map_count=262144
```

Có thể đổi published port trong .env nếu port mặc định bị chiếm. Địa chỉ nội bộ giữa các container không đổi.

5. Khởi động và bootstrap

5.1 Khởi động full stack

```
make down  
make up-full
```

5.2 Bootstrap Elasticsearch

```
docker compose -f docker-compose.full.yml --profile tools run --rm es-init
```

Bootstrap thực hiện:

- Cài index template orders.
- Cài data stream template logs-app-*
- Tạo index orders.
- Áp slow-log settings cho orders hiện có.
- Đăng ký filesystem snapshot repository lab-backups.

5.3 Kiểm tra readiness

```
docker compose -f docker-compose.full.yml ps -a  
curl http://localhost:8000/ready
```

Kết quả đã xác minh:

```
{"status":"ready","checks":{"redis":true,"elasticsearch":true,"rabbitmq":true}}
```

Lưu ý: es-snapshot-init ở trạng thái Exited (0) là đúng thiết kế vì đây là init container.

6. Kiểm tra trạng thái từng thành phần

6.1 RabbitMQ

```
docker compose -f docker-compose.full.yml exec rabbitmq1 rabbitmqctl cluster_status  
docker compose -f docker-compose.full.yml exec rabbitmq1 \  
rabbitmqctl list_queues name type messages_ready messages_unacknowledged
```

Kết quả đã xác minh:

- 3 disk node và 3 running node.
- RabbitMQ 4.1.8, Erlang 27.3.4.12.
- Không alarm.
- Không network partition.
- orders.main.q, orders.retry.q, orders.dlq là quorum queue.
- Tại thời điểm kiểm tra: main và retry queue không backlog; DLQ có 1 message từ bài poison-message test.

6.2 Redis và Sentinel

```
docker compose -f docker-compose.full.yml exec redis redis-cli INFO replication  
docker compose -f docker-compose.full.yml exec redis-sentinel \  
redis-cli -p 26379 SENTINEL master labmaster  
docker compose -f docker-compose.full.yml exec redis-sentinel \  
redis-cli -p 26379 SENTINEL replicas labmaster
```

Kết quả đã xác minh:

- Redis service đang ở role primary.
- Có 1 replica online.
- Sentinel nhận diện primary labmaster và 1 replica.
- Sentinel quorum là 1, chỉ phù hợp lab.

6.3 Elasticsearch

```
curl 'localhost:9200/_cluster/health?pretty'
curl 'localhost:9200/_cat/indices?v'
curl 'localhost:9200/_cat/snapshots/lab-backups?v'
```

Kết quả đã xác minh:

- Elasticsearch một node.
- Index orders green, 1 primary, 0 replica.
- Cluster yellow do backing index logs-app-* hiện có 1 replica nhưng cluster chỉ có một node.
- Snapshot smoke-test SUCCESS, 2/2 shard thành công.

7. Kiểm tra chức năng hệ thống

7.1 Luồng order bình thường

```
curl -i -X POST http://localhost:8000/orders \
-H 'Content-Type: application/json' \
-H 'Idempotency-Key: full-demo-1' \
-H 'X-Trace-ID: full-trace-1' \
-d '{"client_id":"student-1","message":"full lab order","amount":42.5}'
```

Kỳ vọng:

1. API trả HTTP 202 và order_id.
2. API ghi rate-limit/idempotency/job state vào Redis.
3. API publish persistent message vào RabbitMQ.
4. Consumer nhận message và đặt job state processing.
5. Consumer index order vào Elasticsearch.
6. Consumer cập nhật Redis và ACK message.

Kiểm tra sau khi tạo:

```
curl http://localhost:8000/orders/ORDER_ID
curl 'http://localhost:8000/search?q=full'
curl 'http://localhost:8000/logs/search?trace_id=full-trace-1'
```

7.2 Idempotency

```
docker compose -f docker-compose.full.yml --profile tools run --rm client idempotency
```

Kỳ vọng: hai request có cùng idempotency key trả cùng order_id; chỉ một order được publish/xử lý.

7.3 Rate limit

```
docker compose -f docker-compose.full.yml --profile tools run --rm client rate-limit --count 12
```

Kỳ vọng với giới hạn mặc định 10 request/phút: request vượt ngưỡng trả HTTP 429; Redis có key `rate:<client>:<minute>`.

7.4 Retry và dead-letter queue

```
docker compose -f docker-compose.full.yml --profile tools run --rm client dlq
```

Poison order có `simulate_error=true`. Consumer retry qua TTL retry queue; sau `MAX_RETRIES=3`, message vào `orders.dlq`.

7.5 Traceability

```
docker compose -f docker-compose.full.yml --profile tools run --rm client trace
```

Kỳ vọng: API log và consumer log có cùng `trace_id`; Logstash ingest log vào Elasticsearch để truy vấn end-to-end.

8. Kiểm tra hiệu năng

8.1 Phương pháp

Mở hai terminal:

```
# Terminal 1
make perf-full-observe

# Terminal 2
make perf-full-http
```

Observer thu thập theo chu kỳ:

- CPU, RAM, network I/O và block I/O của container.
- RabbitMQ `messages_ready`, `messages_unacknowledged`, `message_bytes`.
- Redis ops/s, eviction, hit/miss, memory và fragmentation.
- Elasticsearch thread-pool `queue/rejected`, indexing và search statistics.

Load generator trả:

- Throughput.
- HTTP status distribution.
- Mean, p50, p95, p99 và max latency.

8.2 HTTP load smoke test đã chạy

Thông số:

- 20 request.

- Concurrency 5.
- Payload 64 byte.

Kết quả:

Chỉ số	Giá trị
HTTP 202	20/20
Throughput	49,57 request/giây
Mean latency	97,50 ms
p50	90,38 ms
p95	154,29 ms
p99	154,29 ms
Max	169,79 ms

Đây là smoke test để xác minh tooling, không phải capacity benchmark.

8.3 Search load smoke test đã chạy

Thông số:

- 20 request.
- Concurrency 5.

Kết quả:

Chỉ số	Giá trị
HTTP 200	20/20
Throughput	144,60 request/giây
Mean latency	29,64 ms
p50	23,15 ms
p95	65,52 ms
p99	65,52 ms
Max	68,06 ms

8.4 Kịch bản capacity test đề xuất

Chạy cùng payload và tăng concurrency theo bậc:

```
docker compose -f docker-compose.full.yml --profile tools run --rm \
  --build --entrypoint python client perf_test.py http \
  --count 10000 --concurrency 25 --payload-bytes 512
```

```
docker compose -f docker-compose.full.yml --profile tools run --rm \
  --build --entrypoint python client perf_test.py http \
  --count 10000 --concurrency 50 --payload-bytes 512
```

```
docker compose -f docker-compose.full.yml --profile tools run --rm \
  --build --entrypoint python client perf_test.py http \
  --count 10000 --concurrency 100 --payload-bytes 512
```

Điểm bão hòa được xác định khi tăng concurrency không tăng throughput nhưng p95/p99, backlog hoặc rejected task tiếp tục tăng.

9. RabbitMQ persistence trade-off

9.1 Phương pháp

```
make perf-full-rabbit
```

Benchmark tạo classic durable queue riêng, bật publisher confirms tuần tự, cùng payload 1.024 byte và so sánh:

- Transient message: delivery_mode=1.
- Persistent message: delivery_mode=2.

Không dùng quorum queue cho phép so sánh này vì quorum queue luôn persist và replicate message qua đa số node.

9.2 Kết quả đã chạy

Thông số: 5.000 message mỗi mode, payload 1.024 byte.

Chỉ số	Transient	Persistent
Throughput	3.125,16 msg/s	2.207,38 msg/s
Mean publish-confirm latency	0,32 ms	0,45 ms
p50	0,20 ms	0,37 ms
p95	0,80 ms	0,76 ms
p99	2,07 ms	1,65 ms
Max	11,10 ms	11,61 ms

Persistent throughput penalty quan sát được: **29,37%**. Transient throughput cao hơn khoảng **1,42 lần**.

Kết luận: persistent message có chi phí throughput do yêu cầu ghi bền trước confirm. Đối lại, message có thể sống sót sau broker restart. Con số cụ thể phụ thuộc disk, fsync, batching, payload và tải nền; cần chạy lặp lại để có kết luận thống kê.

10. Quan sát và phát hiện bottleneck

10.1 RabbitMQ

```
docker compose -f docker-compose.full.yml exec rabbitmq1 \
  rabbitmqctl list_queues name type consumers messages_ready \
  messages_unacknowledged message_bytes memory
docker compose -f docker-compose.full.yml exec rabbitmq1 rabbitmq-diagnostics memory_breakdown
```

Tín hiệu:

- messages_ready tăng liên tục: consumer xử lý chậm hơn publisher.

- messages_unacknowledged cao lâu dài: consumer xử lý chậm hoặc prefetch quá lớn.
- message_bytes, memory và disk I/O tăng: backlog tạo áp lực tài nguyên.
- Connection/channel churn cao: publisher mở quá nhiều connection.
- Quorum queue mất đa số node: queue không thể tiếp tục confirm an toàn.

10.2 Redis

Lab đã bật:

- latency-monitor-threshold = 10ms.
- slowlog-log-slower-than = 1000 microseconds.

Lệnh chẩn đoán:

```
docker compose -f docker-compose.full.yml exec redis redis-cli INFO stats
docker compose -f docker-compose.full.yml exec redis redis-cli INFO memory
docker compose -f docker-compose.full.yml exec redis redis-cli INFO commandstats
docker compose -f docker-compose.full.yml exec redis redis-cli SLOWLOG GET 20
docker compose -f docker-compose.full.yml exec redis redis-cli LATENCY DOCTOR
```

Tín hiệu:

- evicted_keys tăng: thiếu memory.
- Ops/s không tăng nhưng client latency tăng: Redis hoặc CPU có thể bão hòa.
- Slowlog xuất hiện command bất thường: kiểm tra command complexity và data shape.
- Memory fragmentation cao: cần theo dõi allocator và footprint thực.

10.3 Elasticsearch

```
curl 'localhost:9200/_cat/thread_pool/write,search?v'
curl 'localhost:9200/_nodes/stats/indices,indexing_pressure,jvm,fs?pretty'
curl 'localhost:9200/_cat/indices/orders?v'
```

Tại thời điểm kiểm tra, search và write thread pool có active=0, queue=0, rejected=0.

Tín hiệu:

- Thread-pool queue tăng hoặc rejected lớn hơn 0: node không xử lý kịp.
- JVM heap cao và GC kéo dài: heap pressure.
- Disk I/O cao hoặc disk gần đầy: merge, refresh và snapshot cạnh tranh tài nguyên.
- Indexing/search time tăng nhanh hơn operation count: latency mỗi operation xấu đi.

11. Elasticsearch slow logs

Index orders được cấu hình:

- Search query warn: 100ms.
- Search query info: 50ms.
- Search fetch warn: 100ms.
- Indexing warn: 100ms.

- Indexing info: 50ms.

Kiểm tra:

```
curl 'localhost:9200/orders/_settings?pretty&filter_path=*.settings.index.*.slowlog.*'  
docker compose -f docker-compose.full.yml logs -f elasticsearch |  
grep -E 'index.search.slowlog|index.indexing.slowlog'
```

Slow logs giúp xác định query/index operation vượt ngưỡng. Khi thấy một query lặp lại, cần review mapping, query shape, filter, shard count và data distribution trước khi tăng tài nguyên.

12. Elasticsearch backup và restore

12.1 Cấu hình

- Repository: lab-backups.
- Loại: filesystem.
- Vị trí trong container: /mnt/snapshots.
- Docker volume: es-snapshots.
- Có init container đặt quyền volume cho Elasticsearch user.

12.2 Tạo và kiểm tra snapshot

```
make es-snapshot  
make es-snapshot-list  
./scripts/elasticsearch/snapshot.sh status SNAPSHOT_NAME
```

Snapshot smoke-test đã được tạo thành công:

- Trạng thái: SUCCESS.
- Indices: orders và backing index log.
- Successful shards: 2/2.
- Failed shards: 0.

12.3 Restore

```
./scripts/elasticsearch/snapshot.sh restore-orders SNAPSHOT_NAME  
curl 'localhost:9200/orders-restored/_search?pretty'
```

Restore đổi tên thành orders-restored để không ghi đè index đang hoạt động.

Giới hạn: snapshot nằm trong Docker volume cùng host. make clean sẽ xóa volume. Production cần repository độc lập như S3/GCS/Azure hoặc shared filesystem, retention policy, encryption và restore drill định kỳ.

13. Chaos testing và failure scenarios

13.1 Mất một RabbitMQ node

```
docker compose -f docker-compose.full.yml stop rabbitmq2  
curl -X POST http://localhost:8000/orders \  
-H 'Content-Type: application/json' \  
{  
  "id": 1,  
  "type": "order",  
  "value": 100  
}
```

```
-d '{"client_id":"chaos-rabbit","message":"quorum survives","amount":10}'  
docker compose -f docker-compose.full.yml start rabbitmq2
```

Kỳ vọng: quorum queue vẫn hoạt động với 2/3 node. Nếu mất thêm một node, queue mất đa số và không thể tiếp tục đảm bảo ghi an toàn.

13.2 Dừng consumer để tạo backlog

```
docker compose -f docker-compose.full.yml stop consumer-worker  
make perf-full-http  
docker compose -f docker-compose.full.yml exec rabbitmq1 \  
  rabbitmqctl list_queues name messages_ready messages_unacknowledged  
docker compose -f docker-compose.full.yml start consumer-worker
```

Kỳ vọng: messages_ready tăng khi consumer dừng và giảm dần sau khi consumer khởi động lại.

13.3 Redis primary failure

```
docker compose -f docker-compose.full.yml stop redis  
curl -i http://localhost:8000/ready  
sleep 10  
docker compose -f docker-compose.full.yml exec redis-sentinel \  
  redis-cli -p 26379 SENTINEL master labmaster
```

Sentinel có thể promote replica. Tuy nhiên API kết nối trực tiếp tới hostname redis, không dùng Sentinel-aware client, nên ứng dụng không tự chuyển sang primary mới. Đây là giới hạn chủ ý để minh họa yêu cầu production.

13.4 Elasticsearch failure

```
docker compose -f docker-compose.full.yml stop elasticsearch  
curl -i http://localhost:8000/ready  
docker compose -f docker-compose.full.yml start elasticsearch
```

Kỳ vọng: readiness trả 503. Consumer retry rồi đưa message vào DLQ nếu outage kéo dài vượt retry budget.

14. Phát hiện, rủi ro và khuyến nghị

14.1 Phát hiện mức cao

1. **Elasticsearch cluster đang yellow.** Backing index log hiện có một replica nhưng cluster chỉ có một node. Không mất primary data tại thời điểm kiểm tra, nhưng replica không thể allocate.
2. **Redis failover chưa end-to-end.** Sentinel hoạt động nhưng API/worker không dùng Sentinel-aware client.
3. **Chỉ có một Sentinel.** Không đủ khả năng ra quyết định failover an toàn trong production.
4. **Elasticsearch là single-node.** Không chịu được lỗi node và không phản ánh kiến trúc production HA.
5. **Snapshot repository nằm cùng Docker host.** Không bảo vệ trước host failure hoặc make clean.

14.2 Phát hiện mức trung bình

1. API mở một RabbitMQ connection mới cho mỗi request. Khi tải cao, connection churn có thể trở thành bottleneck.

2. Consumer chỉ có một process và mặc định giả lập xử lý 0,25s; backlog sẽ tăng nhanh khi publisher vượt processing capacity.
3. Rate limit và job state cùng nằm trong Redis primary; Redis outage ảnh hưởng trực tiếp API readiness và order creation.
4. Full mode chưa có Prometheus/Grafana dashboard và alert rule hoàn chỉnh; observer hiện là CLI polling.
5. Elasticsearch slow-log threshold thấp phù hợp lab nhưng có thể tạo log volume lớn trong production.

14.3 Khuyến nghị ưu tiên

Ưu tiên	Khuyến nghị	Mục tiêu
P0	Dùng Sentinel-aware Redis client hoặc Redis Cluster	Failover ứng dụng end-to-end
P0	Đưa snapshot repository ra ngoài host và kiểm tra restore định kỳ	Khả năng phục hồi dữ liệu
P1	Chạy Elasticsearch nhiều node/fault domain, sửa replica policy	HA và cluster green
P1	Dùng RabbitMQ connection/channel pool	Giảm connection churn
P1	Thêm Prometheus, Grafana và alert rules	Quan sát liên tục và cảnh báo
P1	Chạy capacity test theo bậc và lặp lại nhiều lần	Xác định SLO/capacity đáng tin cậy
P2	Scale consumer và review prefetch	Kiểm soát backlog và throughput
P2	Thêm restore drill, RTO/RPO và runbook incident	Nâng chất lượng vận hành

15. Kết luận

Full lab đã chứng minh được các khái niệm trọng yếu: durable messaging, quorum queue, retry/DLQ, idempotency, rate limiting, traceability, high-load observation, slow logs, snapshot và failure injection.

Kết quả RabbitMQ benchmark cho thấy persistent message giảm throughput khoảng 29,37% trong lần chạy 5.000 message hiện tại. Đây là trade-off hợp lý khi yêu cầu durability, nhưng cần benchmark lặp lại trên storage và workload gần production.

Lab đạt mục tiêu đào tạo và review kiến trúc. Trước khi xem là production-ready, cần xử lý các khoảng trống HA của Elasticsearch/Redis, đưa backup ra ngoài host, giảm RabbitMQ connection churn và bổ sung monitoring/alerting chuẩn.

Phụ lục A. Lệnh vận hành nhanh

```
# Khởi động
make down
make up-full
docker compose -f docker-compose.full.yml --profile tools run --rm es-init

# Readiness và trạng thái
curl http://localhost:8000/ready
docker compose -f docker-compose.full.yml ps -a

# Quan sát và tải
make perf-full-observe
```

```
make perf-full-http
make perf-full-search
make perf-full-rabbit

# Backup
make es-snapshot
make es-snapshot-list

# Log
docker compose -f docker-compose.full.yml logs -f --tail=100

# Dừng giữ dữ liệu
docker compose -f docker-compose.full.yml down --remove-orphans

# Xóa toàn bộ dữ liệu và snapshot lab
make clean
```

Phụ lục B. Tài liệu liên quan

- README.vi.md: giới thiệu và quick start.
- docs/vi/full-lab.md: runbook full lab từng bước.
- docs/vi/performance.md: hướng dẫn hiệu năng và phát hiện bottleneck.
- docker-compose.full.yml: định nghĩa full stack.
- client/perf_test.py: HTTP/search/RabbitMQ benchmark.
- scripts/perf/observe_full.sh: observer CLI.
- scripts/elasticsearch/snapshot.sh: snapshot và restore.